

Python数据容器



目录

Contents

- ◆ 容器介绍
- ◆ 字符串介绍
- ◆ 字符串下标和切片
- ◆ 字符串常用操作方法
- ◆ 列表介绍
- ◆ 列表的遍历
- ◆ 列表常用操作方法
- ◆ 元组及其应用场景



容器介绍

学习目标

Learning Objectives

1. 知道容器的作用

什么是容器

容器：容器是一种把多个元素组织在一起的**数据结构**，容器中的元素可以**逐个地迭代获取**，可以用各种内置方法对容器中的数据进行**增删改查**等操作

人话：容器就是**存储数据**的东西，同时Python为了方便我们对容器中的数据进行**增加删除修改查询**专门提供了相应的方法便于我们操作



容器的作用

思考

在之前的学习我们知道**变量**也是可以**存储数据**的, 那为什么还要使用**容器存储数据**呢?

解答

变量只能存储**简单的数据**, 比如age=10,name="老王". 这里我们发现普通变量只能存储**单个数据**, 只能存储**单一类型**的数据. 如果我们想存储一个学生的信息, 这时候我们需要存储姓名(字符串类型), 年龄(整数类型), 身高(小数类型)等数据, 那么这个学生的数据实际上是由**多个数据, 多种数据类型**组成的, 普通的变量是满足不了存储需求的, 这时候**容器**就上场了.



Python中的容器

Python中常见容器有如下几种:

- ①字符串
- ②列表
- ③元组
- ④字典

下面我们分别介绍这几种容器





目录

Contents

- ◆ 容器介绍
- ◆ 字符串介绍
- ◆ 字符串下标和切片
- ◆ 字符串常用操作方法
- ◆ 列表介绍
- ◆ 列表的遍历
- ◆ 列表常用操作方法
- ◆ 元组及其应用场景



字符串介绍

学习目标

Learning Objectives

1. 掌握如何定义字符串

字符串的定义

字符串：字符串常常用来存储如文字, 字母等描述性的数据

① 一对引号字符串

```
name1 = 'Tom'  
name2 = "Rose"
```

② 三引号字符串

```
name3 = ''' Tom '''  
name4 = """ Rose """  
  
a = '''I am Tom,  
    nice to meet you! '''  
  
b = """ I am Rose,  
    nice to meet you! """
```

注意：

- 三引号形式的字符串支持换行
- 双引号或者单引号中的数据，就是字符串

字符串的定义



思考：如何创建一个字符串I'm Tom?

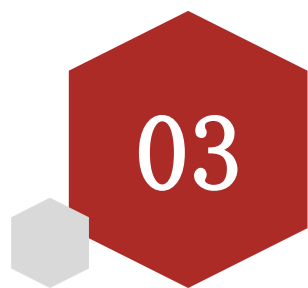
```
Python字符串.py ×
1  str1 = 'I\'m Tom'
2  str2 = "I'm Tom"
3
4  print(str1)
5  print(str2)
```



目录

Contents

- ◆ 容器介绍
- ◆ 字符串介绍
- ◆ 字符串下标和切片
- ◆ 字符串常用操作方法
- ◆ 列表介绍
- ◆ 列表的遍历
- ◆ 列表常用操作方法
- ◆ 元组及其应用场景



字符串的下标和切片

学习目标

Learning Objectives

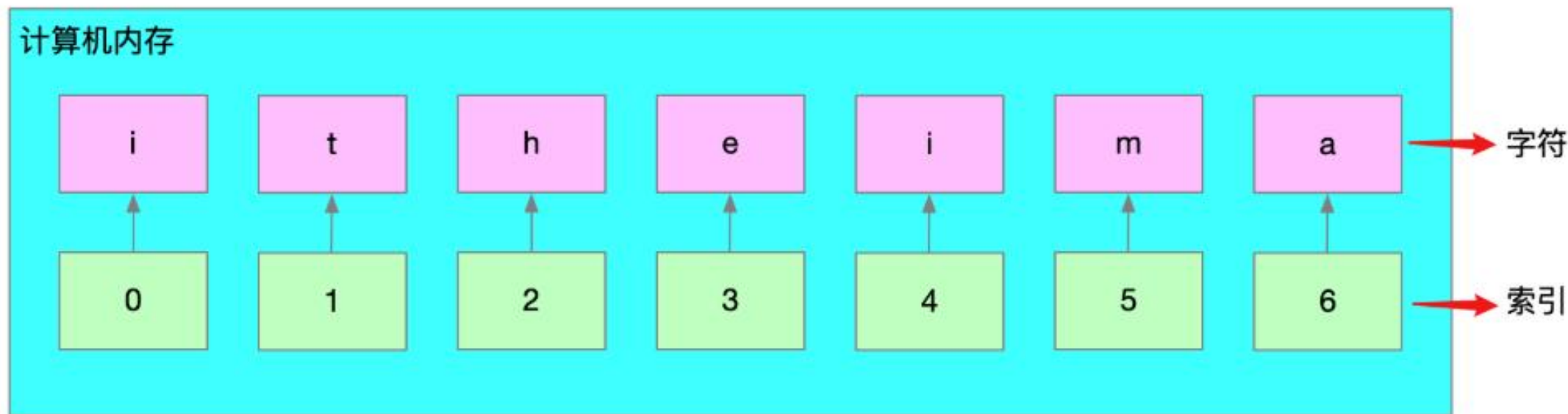
1. 知道什么是下标
2. 掌握切片的使用

字符串的底层存储结构

在计算机底层，Python中的字符串是一段连续的内存地址。

```
str1 = 'itheima'
```

底层存储结构：



注意：索引下标从0开始

聊聊索引下标

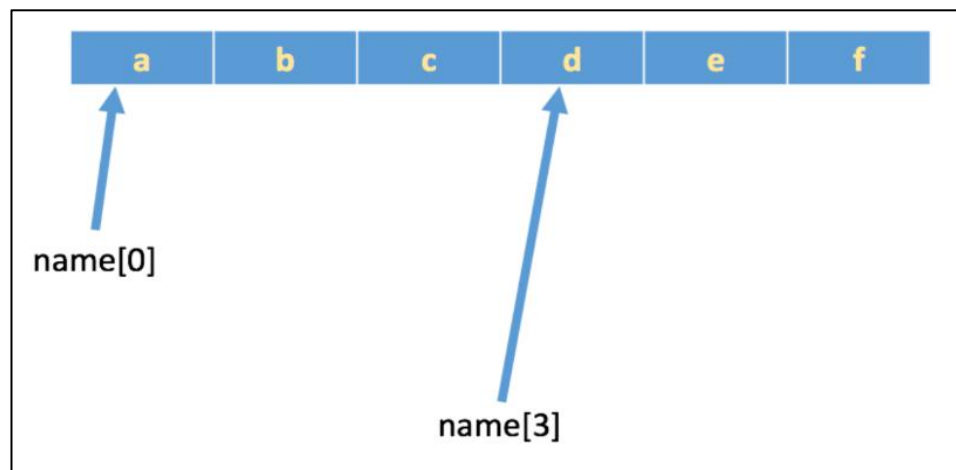
‘索引下标’，就是编号。比如火车座位号，座位号的作用：按照编号快速找到对应的座位

同理，下标的作用即是通过下标快速找到对应的数据



聊聊索引下标

如果有字符串: `name = 'abcdef'`，在内存中的实际存储如下：



如果想取出部分字符，那么可以通过下标的方法（注意python中下标从 0 开始）

```
name = 'abcdef'
```

```
print(name[0])
```

```
print(name[1])
```

```
print(name[2])
```

a

b

c

注意：

- 列表与元组也同样支持下标操作
- 下标也叫做索引



字符串的切片

基本语法

切片是指对操作的对象截取其中一部分的操作。比如想要取字符串"888666@qq.com"中的QQ号的时候就可以使用切片**字符串、列表、元组**都支持切片操作

切片的语法：[起始:结束:步长]

注意：选取的区间从"起始"位开始，到"结束"位的前一位结束（不包含结束位本身），步长表示选取间隔的长度

	→ 从左往右					← 从右往左				
正索引	0	1	2	3	4	5	6	7	8	9
负索引	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
值	0	1	2	3	4	5	6	7	8	9
	↑ 起点									↑ 终点

切片的使用

记口诀：切片其实很简单，只顾头来尾不管，步长为正正向移，步长为负则逆向移

```
字符串切片.py ×
1  name = "abcdefg"
2
3  print(name[2:5:1])  # cde
4  print(name[2:5])    # cde
5  print(name[:5])     # abcde
6  print(name[1:])     # bcdefg
7  print(name[:])      # abcdefg
8  print(name[:2])     # aceg
9  print(name[:-1])    # abcdef, 负1表示倒数第一个数据
10 print(name[-4:-1])  # def
11 print(name[::-1])   # gfedcba
```

练一练

```
s = 'Hello World!'

print(s[4])

print(s)

print(s[:]) # 取出所有元素（没有起始位和结束位之分），默认步长为1

print(s[1:]) # 从下标为1开始，取出 后面所有的元素（没有结束位）

print(s[:5]) # 从起始位置开始，取到 下标为5的前一个元素（不包括结束位本身）

print(s[:-1]) # 从起始位置开始，取到 倒数第一个元素（不包括结束位本身）

print(s[-4:-1]) # 从倒数第4个元素开始，取到 倒数第1个元素（不包括结束位本身）

print(s[1:5:2]) # 从下标为1开始，取到下标为5的前一个元素，步长为2（不包括结束位本身）
```

切片的使用

（面试题）给定一个字符串aStr，请反转字符串

python 字符串快速逆置 `print(s[::-1])` # 从后向前，按步长为1进行取值



目录

Contents

- ◆ 容器介绍
- ◆ 字符串介绍
- ◆ 字符串下标和切片
- ◆ 字符串常用操作方法
- ◆ 列表介绍
- ◆ 列表的遍历
- ◆ 列表常用操作方法
- ◆ 元组及其应用场景



字符串常用操作方法

学习目标

Learning Objectives

1. 掌握查找方法`find()`, `index()`
2. 掌握修改方法`replace()`, `split()`

字符串查找

所谓字符串查找方法即是查找子串在字符串中的位置或出现的次数

字符串查找

`find()`：检测某个子串是否包含在这个字符串中，如果在返回这个子串开始的位置下标，否则则返回-1。

基本语法：

字符串序列.`find`(子串, 开始位置下标, 结束位置下标)

注意：开始和结束位置下标可以省略，表示在整个字符串序列中查找。

案例演示：

```
mystr = "hello world and hello python"
```

```
print(mystr.find('hello')) # 0
```

```
print(mystr.find('hello', 10, 27)) # 16
```

```
print(mystr.find('heima')) # -1
```

字符串查找

`index()`：检测某个子串是否包含在这个字符串中，如果在返回这个子串开始的位置下标，否则则报异常。

基本语法：

字符串序列.`index`(子串, 开始位置下标, 结束位置下标)

注意：开始和结束位置下标可以省略，表示在整个字符串序列中查找。

案例演示：

```
mystr = "hello world and hello python"
```

```
print(mystr.index('hello')) # 0
```

```
print(mystr.index('hello', 10, 27)) #16
```

```
print(mystr.index('heima')) # 报错
```

字符串修改

所谓修改字符串，指的就是通过函数的形式修改字符串中的数据

字符串修改

`replace()`：返回替换后的字符串

基本语法：

字符串序列.`replace`(旧子串, 新子串, 替换次数)

注意：替换次数如果查出子串出现次数，则替换次数为该子串出现次数。

案例演示：

```
mystr = "hello world and hello python"
```

```
# 结果: hello world & hello python
```

```
print(mystr.replace('and', '&'))
```

```
# 结果: hi world and hi python
```

```
print(mystr.replace('hello', 'hi', 2))
```

```
# 结果: hello world and hello python
```

```
print(mystr)
```

注意：数据按照是否能直接修改分为可变类型和不可变类型两种。由于字符串属于不可变类型，所以修改的时候不能改变原有字符串。

字符串修改

`split()`：按照指定字符分割字符串。

基本语法：

字符串序列.`split`(分割字符, num)

注意：num表示的是分割字符出现的次数，即将来返回数据个数为num+1个。

案例演示：

```
mystr = 'apple,orange,banana'
```

```
# 结果：['apple', 'orange', 'banana']  
print(mystr.split(','))
```

```
# 结果：['apple', 'orange,banana']  
print(mystr.split(',', 1))
```

```
# 结果：['apple,orange,banana']  
print(mystr.split('*'))
```


字符串操作方法

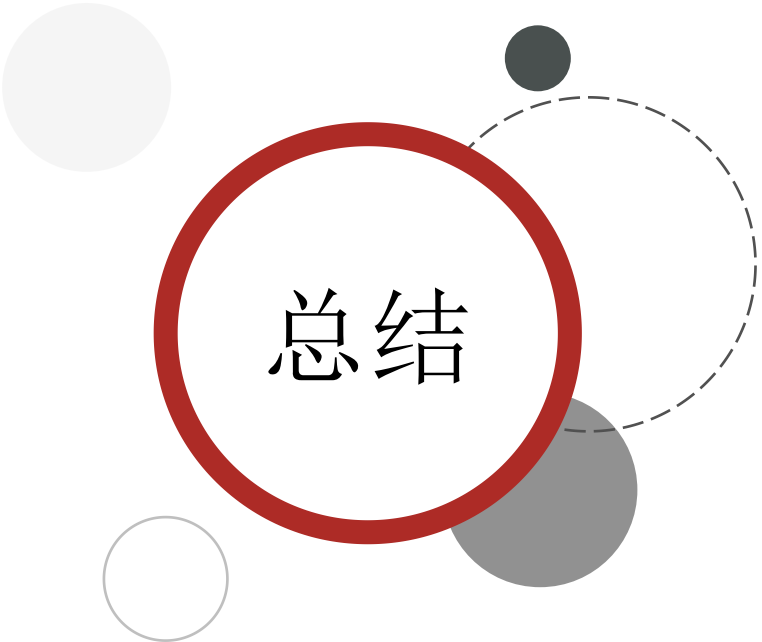
编号	函数	作用
1	find()	检测某个子串是否包含在这个字符串中，如果在返回这个子串开始的位置下标，否则则返回-1。
2	index()	检测某个子串是否包含在这个字符串中，如果在返回这个子串开始的位置下标，否则则报异常。
3	replace()	返回替换后的字符串
4	split()	返回切割后的列表序列

练习

Python字符串练习题

练习题1：输入一个字符串，打印所有偶数位上的字符(下标是0，2，4，6...位上的字符)

练习题2：给定一个文件名，判断其尾部是否以".png"结尾



总结

常用查找操作方法

- `find()`
- `index()`

常用修改操作方法

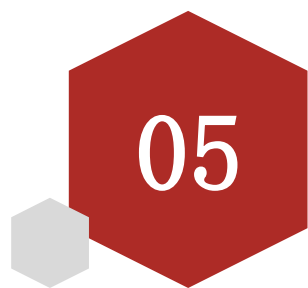
- `replace()`
- `split()`



目录

Contents

- ◆ 容器介绍
- ◆ 字符串介绍
- ◆ 字符串下标和切片
- ◆ 字符串常用操作方法
- ◆ 列表介绍
- ◆ 列表的遍历
- ◆ 列表常用操作方法
- ◆ 元组及其应用场景



列表介绍

学习目标

Learning Objectives

1. 掌握列表的基本格式

为什么需要列表

思考：有一个人的姓名(TOM)怎么书写存储程序？

答：变量

思考：如果一个班级100位学生，每个人的姓名都要存储，应该如何书写程序？声明100个变量吗？

答：No，我们使用列表就可以了， 列表一次可以存储多个数据

列表的定义

基本语法:

```
列表名称 = [数据1, 数据2, 数据3, 数据4, ...]
```

案例演示:

```
name_list = ['Tom', 'Lily', 'Rose']  
print(name_list)  
print(type(name_list))
```

结果:

```
['Tom', 'Lily', 'Rose']  
<class 'list'>
```

注意事项

注意：列表可以一次存储多个数据且可以为不同的数据类型

列表的下标

索引下标

```
name_list = ['Tom', 'Lily', 'Rose']  
  
print(name_list[0]) # Tom  
print(name_list[1]) # Lily  
print(name_list[2]) # Rose
```

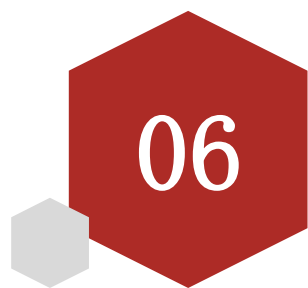
看起来是不是特别熟悉，列表的底层存储形式和字符串都是一样的，其也是通过索引下标来对其进行引用的



目录

Contents

- ◆ 容器介绍
- ◆ 字符串介绍
- ◆ 字符串下标和切片
- ◆ 字符串常用操作方法
- ◆ 列表介绍
- ◆ 列表的遍历
- ◆ 列表常用操作方法
- ◆ 元组及其应用场景



列表的遍历

学习目标

Learning Objectives

1. 掌握列表的循环怎么使用

列表的遍历

如果想要更有效率的输出列表的每个数据，可以使用循环来完成，使用循环把列表中的每个数据输出一遍的

过程叫做遍历

使用for循环

```
names_list = ['xiaoWang', 'xiaoZhang', 'xiaoHua']  
for name in names_list:  
    print(name)
```

结果:

```
xiaoWang  
xiaoZhang  
xiaoHua
```

列表的遍历

如果想要更有效率的输出列表的每个数据，可以使用循环来完成，使用循环把列表中的每个数据输出一遍的

过程叫做遍历

使用while循环

```
names_list = ['xiaoWang', 'xiaoZhang', 'xiaoHua']

length = len(names_list)

i = 0

while i < length:
    print(names_list[i])
    i += 1
```

结果:

```
xiaoWang
xiaoZhang
xiaoHua
```



目录

Contents

- ◆ 容器介绍
- ◆ 字符串介绍
- ◆ 字符串下标和切片
- ◆ 字符串常用操作方法
- ◆ 列表介绍
- ◆ 列表的遍历
- ◆ 列表常用操作方法
- ◆ 元组及其应用场景



列表常用操作方法

学习目标

Learning Objectives

1. 掌握列表的“增”、“删”、“改”、“查”操作

列表的常用操作

列表的作用是一次性存储多个数据，程序员可以对这些数据进行的操作有：增、删、改、查

添加元素 ("增"append, extend, insert)

修改元素 ("改",通过下标修改)

查找元素 ("查"in, not in, index, count)

删除元素 ("删"del, pop, remove)

排序 (sort, reverse)

☆ 增加

append() 方法：增加指定数据到列表中

```
name_list = ['Tom', 'Lily', 'Rose']  
name_list.append('Jennifer')  
  
print(name_list)
```

```
Run: Python列表 ×  
▶ ↑ D:\Python3\pythonProject\venv\Scripts\python.exe D:/Python3/pythonProject/Python列表.py  
■ ↓ ['Tom', 'Lily', 'Rose', 'Jennifer']  
⏏ ↻ Process finished with exit code 0
```

注意事项

注意：列表追加数据的时候，直接在原列表里面追加了指定数据，即修改了原列表，故列表为可变类型数据。

☆ 增加

`extend()` 方法：列表结尾追加数据，如果数据是一个序列，则将这个序列的数据逐一添加到列表

列表序列.extend(数据)

单个数据演示：

```
name_list = ['Tom', 'Lily', 'Rose']
name_list.extend('Smith')

# 结果：['Tom', 'Lily', 'Rose', 'S', 'm', 'i', 't', 'h']
print(name_list)
```

序列数据演示：

```
name_list = ['Tom', 'Lily', 'Rose']
name_list.extend(['Smith', 'Jennifer'])

# 结果：['Tom', 'Lily', 'Rose', 'Smith', 'Jennifer']
print(name_list)
```

☆ 增加

`insert()` 方法：指定位置新增数据

列表序列.insert(位置下标, 数据)

案例演示：

```
name_list = ['Tom', 'Lily', 'Rose']  
name_list.insert(1, 'Smith')  
  
# 结果： ['Tom', 'Smith', 'Lily', 'Rose']  
print(name_list)
```

☆ 增加

编号	函数	作用
1	<code>append()</code>	增加指定数据到列表中
2	<code>extend()</code>	列表结尾追加数据，如果数据是一个序列，则将这个序列的数据逐一添加到列表
3	<code>insert()</code>	指定位置新增数据

☆ 查询

`index()` 方法：指定数据所在位置的下标

列表序列.index(数据, 开始位置下标, 结束位置下标)

案例演示：

```
name_list = ['Tom', 'Lily', 'Rose']  
  
print(name_list.index('Lily', 0, 2)) # 1
```

☆ 查询

`count()` 方法：统计指定数据在当前列表中出现的次数

```
列表序列.count(数据)
```

案例演示：

```
name_list = ['Tom', 'Lily', 'Rose']  
  
print(name_list.count('Lily')) # 1
```


☆ 查询

in: 判断指定数据在某个列表序列，如果在返回True，否则返回False

```
name_list = ['Tom', 'Lily', 'Rose']
```

```
# 结果: True
```

```
print('Lily' in name_list)
```

```
# 结果: False
```

```
print('Lilys' in name_list)
```

not in: 判断指定数据不在某个列表序列，如果不在返回True，否则返回False

```
name_list = ['Tom', 'Lily', 'Rose']
```

```
# 结果: False
```

```
print('Lily' not in name_list)
```

```
# 结果: True
```

```
print('Lilys' not in name_list)
```

☆ 查询

编号	函数	作用
1	index()	指定数据所在位置的下标
2	count()	统计指定数据在当前列表中出现的次数
3	in	判断指定数据在某个列表序列，如果在返回True，否则返回False
4	not in	判断指定数据不在某个列表序列，如果不在返回True，否则返回False

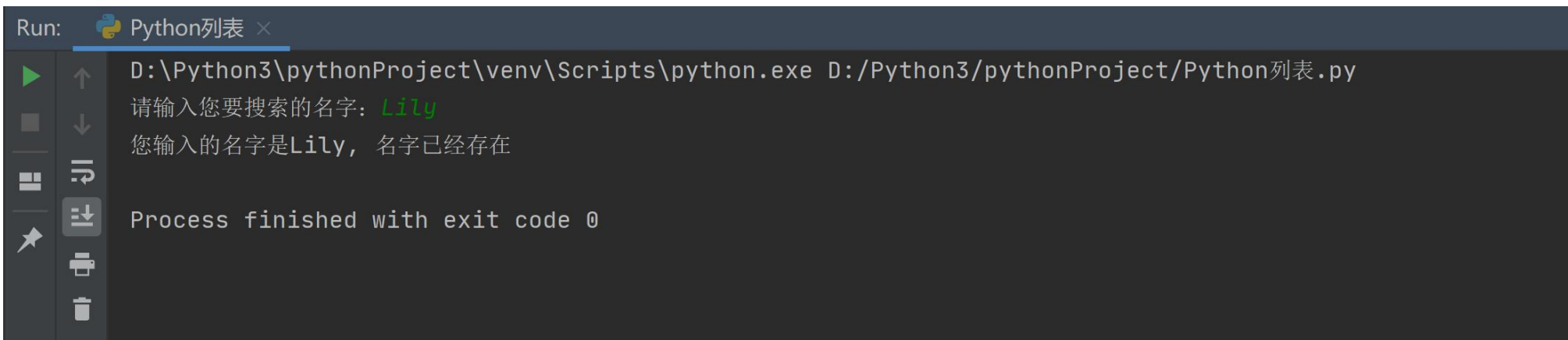
☆ 查询

综合案例

```
name_list = ['Tom', 'Lily', 'Rose']

name = input('请输入您要搜索的名字: ')

if name in name_list:
    print(f'您输入的名字是{name}, 名字已经存在')
else:
    print(f'您输入的名字是{name}, 名字不存在')
```



The screenshot shows a Python IDE window titled "Run: Python列表". The command line shows the execution of a Python script: `D:\Python3\pythonProject\venv\Scripts\python.exe D:/Python3/pythonProject/Python列表.py`. The output shows the user inputting "Lily" and receiving the message "您输入的名字是Lily, 名字已经存在". The process finished with exit code 0.

```
Run: Python列表 ×
D:\Python3\pythonProject\venv\Scripts\python.exe D:/Python3/pythonProject/Python列表.py
请输入您要搜索的名字: Lily
您输入的名字是Lily, 名字已经存在
Process finished with exit code 0
```

☆ 删除

del: 删除列表中的某个元素

del 目标

删除列表，案例演示：

```
name_list = ['Tom', 'Lily', 'Rose']
```

```
del name_list[0]
```

```
print(name_list)
```

☆ 删除

`pop()` 方法：删除指定下标的数据(默认为最后一个)，并返回该数据。

列表序列.`pop`(下标)

案例演示：

```
name_list = ['Tom', 'Lily', 'Rose']

del_name = name_list.pop(1)

# 结果： Lily
print(del_name)

# 结果： ['Tom', 'Rose']
print(name_list)
```

☆ 删除

`remove()` 方法：移除列表中某个数据的第一个匹配项。

```
列表序列.remove(数据)
```

案例演示：

```
name_list = ['Tom', 'Lily', 'Rose']
```

```
name_list.remove('Rose')
```

```
# 结果：['Tom', 'Lily']
```

```
print(name_list)
```

☆ 删除

`clear()` 方法：清空列表，删除列表中的所有元素，返回空列表。

列表序列.`clear()`

案例演示：

```
name_list = ['Tom', 'Lily', 'Rose']
```

```
name_list.clear()
```

```
print(name_list) # 结果： []
```

☆ 删除

编号	函数	作用
1	del 列表[索引]	删除列表中的某个元素
2	pop()	删除指定下标的数据(默认为最后一个)，并返回该数据
3	remove()	移除列表中某个数据的第一个匹配项。
4	clear()	清空列表，删除列表中的所有元素，返回空列表。

☆ 修改

修改指定的元素值

列表[索引] = 修改后的值

案例演示：

```
name_list = ['Tom', 'Lily', 'Rose']
```

```
name_list[0] = 'Smith'
```

```
# 结果： ['Smith', 'Lily', 'Rose']
```

```
print(name_list)
```

☆ 修改

`reverse()`：将数据序列进行倒叙排列

```
列表序列.reverse()
```

案例演示：

```
num_list = [1, 5, 2, 3, 6, 8]
```

```
num_list.reverse()
```

```
# 结果： [8, 6, 3, 2, 5, 1]
```

```
print(num_list)
```

☆ 修改

sort() 方法：对列表序列进行排序

列表序列.sort(key=None, reverse=False)

注意：reverse表示排序规则，reverse = True降序，reverse = False升序（默认）

案例演示：

```
num_list = [1, 5, 2, 3, 6, 8]
```

```
num_list.sort()
```

```
# 结果：[1, 2, 3, 5, 6, 8]
```

```
print(num_list)
```

☆ 修改

sort() 方法：对列表序列进行排序

列表序列.sort(key=None, reverse=False)

注意：reverse表示排序规则，reverse = True降序， reverse = False升序（默认）

案例演示：

```
num_list = [1, 5, 2, 3, 6, 8]
```

```
num_list.sort(reverse = True)
```

```
# 结果： [8, 6, 5, 3, 2, 1]
```

```
print(num_list)
```

☆ 修改

编号	函数	作用
1	列表[索引] = 修改后的值	修改列表中的某个元素
2	reverse()	将数据序列进行倒叙排列
3	sort()	对列表序列进行排序



列表的嵌套

所谓列表嵌套指的就是一个列表里面包含了其他的子列表。

应用场景：要存储班级一、二、三三个班级学生姓名，且每个班级的学生姓名在一个列表。

```
name_list = [['小明', '小红', '小绿'], ['Tom', 'Lily', 'Rose'], ['张三', '李四', '王五']]
```

思考： 如何查找到数据“李四”？

```
# 第一步：按下标查找到李四所在的列表
```

```
print(name_list[2])
```

```
# 第二步：从李四所在的列表里面，再按下标找到数据李四
```

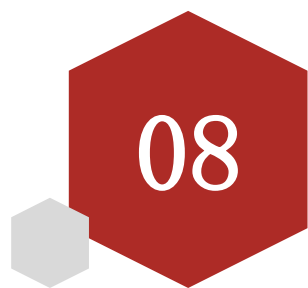
```
print(name_list[2][1])
```



目录

Contents

- ◆ 容器介绍
- ◆ 字符串介绍
- ◆ 字符串下标和切片
- ◆ 字符串常用操作方法
- ◆ 列表介绍
- ◆ 列表的遍历
- ◆ 列表常用操作方法
- ◆ 元组及其应用场景



元组及其应用场景

学习目标

Learning Objectives

1. 元组的应用场景
2. 定义元组
3. 元组常见操作

为什么需要元组

思考：如果想要存储多个数据，但是这些数据是不能修改的数据，怎么做？

答：列表？列表可以一次性存储多个数据，但是列表中的数据允许更改。

```
num_list = [10, 20, 30]  
num_list[0] = 100
```

那这种情况下，我们想要存储多个数据且数据不允许更改，应该怎么办呢？

答：使用元组，元组可以存储多个数据且元组内的数据是不能修改的。

定义元组

元组特点：定义元组使用**小括号**，且使用**逗号**隔开各个数据，**数据可以是不同的数据类型**。

```
# 多个数据元组
```

```
t1 = (10, 20, 30)
```

```
# 单个数据元组
```

```
t2 = (10,)
```

注意：如果定义的元组只有一个数据，那么这个数据后面也要添加逗号，否则数据类型为唯一的这个数据的数据类型。

```
t2 = (10,)
```

```
print(type(t2)) # tuple
```

```
t3 = (20)
```

```
print(type(t3)) # int
```

```
t4 = ('hello')
```

```
print(type(t4)) # str
```

元组的相关操作

编号	函数	作用
1	元组[索引]	根据索引下标查找元素
2	index()	查找某个数据，如果数据存在返回对应的下标，否则报错，语法和列表、字符串的index方法相同
3	count()	统计某个数据在当前元组出现的次数
4	len()	统计元组中数据的个数

元组内的数据如果直接修改则立即报错，但是如果元组里面有列表，修改列表里面的数据则是支持的，故这个特点非常重要。

```
tuple2 = (10, 20, ['aa', 'bb', 'cc'], 50, 30)
print(tuple2[2]) # 访问到列表

# 结果: (10, 20, ['aaaaa', 'bb', 'cc'], 50, 30)
tuple2[2][0] = 'aaaaa'
print(tuple2)
```

元组的相关操作演示

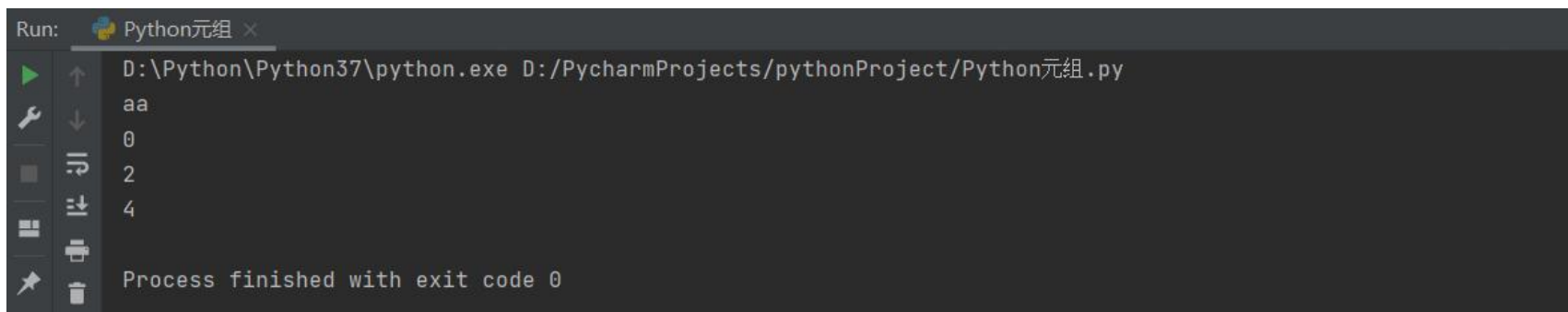
```
tuple1 = ('aa', 'bb', 'cc', 'bb')  
print(tuple1[0]) # aa
```

```
tuple2 = ('aa', 'bb', 'cc', 'bb')  
print(tuple2.index('aa')) # 0
```

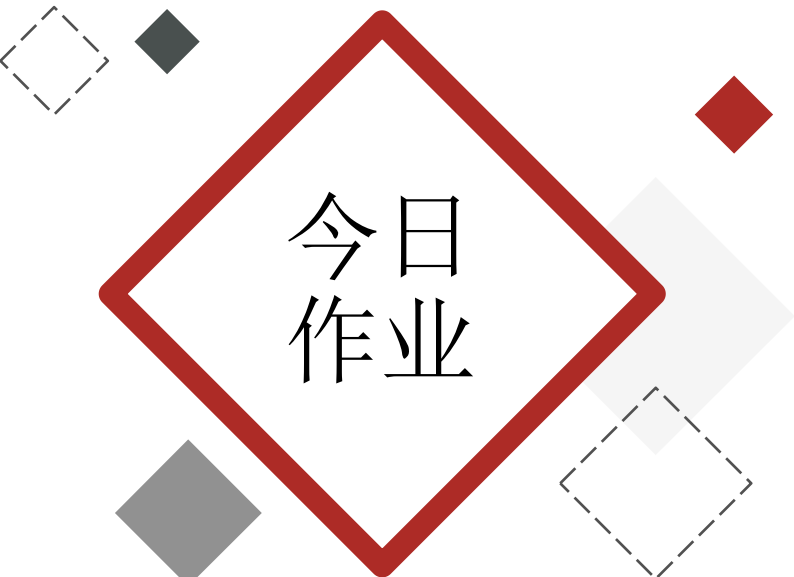
```
tuple3 = ('aa', 'bb', 'cc', 'bb')  
print(tuple3.count('bb')) # 2
```

```
tuple4 = ('aa', 'bb', 'cc', 'bb')  
print(len(tuple4)) # 4
```

运行结果：



```
Run: Python元组 x  
D:\Python\Python37\python.exe D:/PycharmProjects/pythonProject/Python元组.py  
aa  
0  
2  
4  
Process finished with exit code 0
```



今日作业

1. 幸运数字6: 输入任意数字，如数字8，生成nums列表，元素值为1~8，从中选取幸运数字(能够被6整除)移动到新列表lucky，打印nums与lucky。
2. 列表嵌套: 有3个教室[[], [], []], 8名讲师['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H'], 将8名讲师随机分配到3个教室中



传智教育旗下高端IT教育品牌